Chapter 13 (The Abstraction: Address Spaces) — Homework (Code)

Zhiyuan Pan

Code: `code/chapter13/memory-user.c`. Build with `make`, run as `./memory-user <megabytes> [seconds]`.

Test machine: Ubuntu 24.04 VM (VirtualBox on an Apple Silicon Mac), Linux aarch64, 8 GB of RAM, **no swap**, `free` and `pmap` from procps-ng 4.0.4.

Q1

> The first Linux tool you should check out is the very simple tool `free`. First, type `man free` and read its entire manual page; it's short, don't worry!

`free` prints how much physical memory and swap is in use. It reads the numbers from `/proc/meminfo`. The columns:

column	meaning
`total`	usable physical memory (RAM minus a few reserved bits and the kernel binary)
`used`	memory in use, computed as `total − available`
`free`	memory not used for anything at all
`shared`	memory used (mostly) by tmpfs
`buff/cache`	memory the kernel uses for buffers and the page cache (cached file data); it can be given back to programs when needed
`available`	estimate of how much memory new programs can get **without swapping**; it counts `free` plus the part of `buff/cache` that can be reclaimed

Useful options: `-m` / `-g` (show in MB / GB), `-h` (human readable), `-s N` (repeat every N seconds), `-t` (add a total line), `-w` (show buffers and cache in separate columns).

Q2

> Now, run `free`, perhaps using some of the arguments that might be useful (e.g., `-m`, to display memory totals in megabytes). How much memory is in your system? How much is free? Do these numbers match your intuition?

```
$ free -m
               total        used        free      shared  buff/cache   available
Mem:            8196        1167        5625          34        1600        7028
Swap:              0           0           0
```

- **Total: 8196 MB (8 GB)**, which is exactly the memory I gave the VM. There is no swap.
- **Free: 5625 MB**, but **available: 7028 MB**.
- **Used: 1167 MB**, mostly the GNOME desktop (`gnome-shell` alone uses about 410 MB, see Q6).

This mostly matches my intuition (an 8 GB machine running a desktop and little else), with one surprise: `free` is much smaller than `available`. The difference is the **1600 MB of buff/cache**. Linux uses otherwise idle memory to cache file data (programs, libraries, files read recently), because unused RAM is wasted RAM. That cache is given back immediately when a program needs memory, so `available` (7 GB) is the number that answers "how much more can I use?", not `free`.

Q3

> Next, create a little program that uses a certain amount of memory, called `memory-user.c`. This program should take one command-line argument: the number of megabytes of memory it will use. When run, it should allocate an array, and constantly stream through the array, touching each entry. The program should do this indefinitely, or, perhaps, for a certain amount of time also specified at the command line.

`memory-user.c` takes the number of megabytes and an optional number of seconds (without it, it runs until killed):

```
size_t bytes = (size_t) mb * 1024 * 1024;
char *array = malloc(bytes);
...
printf("pid %d: using %ld MB (%zu bytes)\n", (int) getpid(), mb, bytes);
while (seconds < 0 || time(NULL) < end) {
    for (size_t i = 0; i < bytes; i += 4096)   // one write per page is enough
        array[i] = (char) passes;              // to keep every page resident
    passes++;
}
```

Two design points:

- It **writes** to the array, it does not just allocate it. `malloc` only reserves virtual address space; Linux gives the process a physical page only the first time that page is touched. A program that called `malloc` and never touched the memory would barely show up in `free` at all.
- Touching one byte per 4 KB page is enough to make every page resident, and it lets the loop make many passes per second. The program prints its PID to make Q6–Q8 easier.

Q4

> Now, while running your `memory-user` program, also (in a different terminal window, but on the same machine) run the `free` tool. How do the memory usage totals change when your program is running? How about when you kill the `memory-user` program? Do the numbers match your expectations? Try this for different amounts of memory usage. What happens when you use really large amounts of memory?

I ran `./memory-user N 8` and ran `free -m` before it, 5 seconds into the run, and 1 second after it exited. I also read the program's resident set size (`VmRSS` in `/proc/PID/status`) while it was running:

N (MB)	used before	used while running	used after	change	free while running	VmRSS of memory-user
500	1165	1689	1251	**+524**	5623 → 5098	501 MB
1000	1251	2237	1252	**+986**	5536 → 4550	1001 MB
2000	1252	3226	1245	**+1974**	5535 → 3561	2001 MB
4000	1246	5245	1273	**+3999**	5541 → 1541	4002 MB

- **While the program runs, `used` grows (and `free` and `available` shrink) by almost exactly the amount it allocated.** `buff/cache` stays at 1616 MB: the kernel had enough free memory, so it did not need to drop any cache.
- **When the program exits (or is killed), the memory comes back right away**: `used` returns to about 1250 MB. The OS reclaims all of a process's pages when it ends, even though `memory-user` never calls `free()` when it is killed.
- The numbers match my expectations. The small differences (e.g. +524 for 500 MB) are other programs on the desktop changing a little between measurements.

Really large amounts of memory. The VM has 8 GB and no swap, so there is nowhere to put pages that do not fit in RAM. I tested this in two safe ways (filling the whole RAM could get the desktop itself killed):

1. **Asking for far more than exists:**

   ```
   $ ./memory-user 20000 3
   malloc: Cannot allocate memory
   ```

   With the default overcommit policy (`/proc/sys/vm/overcommit_memory = 0`), Linux refuses an allocation that is obviously larger than RAM + swap, so `malloc` returns NULL and the program exits without running.

2. **Asking for more than is allowed, but in smaller steps:** run a 2 GB `memory-user` inside a control group limited to 1 GB:

```
$ systemd-run --user --scope -p MemoryMax=1G -p MemorySwapMax=0 ./memory-user 200
0 5
pid 4393: using 2000 MB (2097152000 bytes)
Killed                                          (exit status 137)

kernel: Memory cgroup out of memory: Killed process 4393 (memory-user)
        total-vm:2050268kB, anon-rss:1046452kB
```

This time `malloc` succeeds, because it only reserves address space. The program starts touching pages, and when it reaches the 1 GB limit (`anon-rss` 1046452 kB), the kernel's **OOM killer** kills it with SIGKILL.

The same thing happens system-wide: if a program touches more memory than RAM plus swap, the OOM killer picks a process (usually the biggest) and kills it. With swap, the system would first start paging memory out to disk and slow down dramatically.

Q5

> Let's try one more tool, known as `pmap`. Spend some time, and read the `pmap` manual page in detail.

`pmap PID` prints the **memory map** of a process: one line per mapping (a contiguous range of virtual addresses with the same permissions and backing). Useful options:

option	output
(none)	address, size, permissions (`r`, `w`, `x`, `p`/`s`) and what is mapped
`-x`	extended: adds RSS (how much is actually in RAM) and Dirty (modified pages)
`-X` / `-XX`	even more detail, from `/proc/PID/smaps` (PSS, swap, huge pages, …)
`-d`	device format: offset into the file and device
`-q`	quiet: no header and footer

Q6

> To use `pmap`, you have to know the process ID of the process you're interested in. Thus, first run `ps auxw` to see a list of all processes; then, pick an interesting one, such as a browser. You can also use your `memory-user` program in this case (indeed, you can even have that program call `getpid()` and print out its PID for your convenience).

`ps auxw --sort=-rss` lists processes, largest resident memory first:

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
zhiyuan     2974  2.2  4.9 4920904 418996 ?      Ssl  11:57   0:22 /usr/bin/gnome-shell
zhiyuan     3840  0.8  3.5 2458568 302108 ?      Sl   11:58   0:08 /usr/bin/nautilus
zhiyuan     3613  0.0  1.2 1424324 102972 ?      Sl   11:57   0:00 /usr/libexec/mutter-
x11-frames
```

`VSZ` is the size of the virtual address space and `RSS` the part that is in physical memory, both in KB. I picked **`gnome-shell`** (PID 2974), the desktop shell and the biggest process. `memory-user` prints its own PID with `getpid()`.

Q7

> Now run `pmap` on some of these processes, using various flags (like `-X`) to reveal many details about the process. What do you see? How many different entities make up a modern address space, as opposed to our simple conception of code/stack/heap?

```
$ pmap -x 2974 | tail -1
total kB         4920904  418996  283356
$ pmap -q 2974 | wc -l
1303
```

`gnome-shell`'s address space consists of **about 1,300 separate mappings** (the count changes a little as the process runs), with 4.9 GB of virtual address space but only 419 MB resident. Instead of three pieces (code, heap, stack), a modern address space contains:

- the **program's own code, read-only data and data**, each a separate mapping with different permissions (`r-x`, `r--`, `rw-`);
- **shared libraries**: **210 different `.so` files** (`libc.so.6`, `libz`, `libxml2`, `libX…`, GTK, the JavaScript engine …). Each one typically appears as **4 mappings** (code, a no-access guard region, read-only data, writable data), e.g. `libc.so.6` has exactly 4;
- **anonymous memory** (`[ anon ]`, **342 mappings**): the heap, plus large `malloc` regions and areas created by `mmap`;
- **stacks**: `[ stack ]` for the main thread, and anonymous regions for each extra thread;
- **memory-mapped data files**: font caches (`*.cache-9`), GObject type libraries (`*.typelib`), and shared-memory buffers used with the display server (27 mappings are marked `(deleted)`: files that were unlinked but are still mapped);
- small kernel-provided regions, `[vvar]` and **`[vdso]`** (visible in `/proc/PID/maps`; the vDSO is the code that makes `gettimeofday` fast, see chapter 6).

Many of these are **shared** between processes (the same `libc.so.6` code is in RAM once for all programs), which is why the sum of all processes' RSS is larger than `used` in `free`.

Q8

> Finally, let's run `pmap` on your `memory-user` program, with different amounts of used memory. What do you see here? Does the output from `pmap` match your expectations?

`./memory-user 100 6`, then `pmap PID`:

```
0000c768a1440000      4K r-x-- memory-user        ← code
0000c768a145f000      4K r---- memory-user        ← read-only data
0000c768a1460000      4K rw--- memory-user        ← data / globals
0000c768a15d3000    132K rw---   [ anon ]         ← heap (brk area)
0000f42b8479f000 102404K rw---   [ anon ]         ← the 100 MB array
0000f42b8aba0000   1640K r-x-- libc.so.6          ← C library (4 mappings)
0000f42b8ad3a000     76K ----- libc.so.6
0000f42b8ad4d000     12K r---- libc.so.6
0000f42b8ad50000      8K rw--- libc.so.6
0000f42b8ad52000     48K rw---   [ anon ]
0000f42b8ad62000    156K r-x-- ld-linux-aarch64.so.1   ← dynamic loader
...
0000ffffdb680000    132K rw---   [ stack ]
 total           104668K
```

With `pmap -x`, the biggest mappings and the totals:

run	array mapping	its RSS	total virtual	total RSS
100 MB	102,404 KB `[ anon ]`	102,400 KB	104,668 KB	103,932 KB
1000 MB	1,024,004 KB `[ anon ]`	1,024,000 KB	1,026,268 KB	1,025,532 KB

This matches my expectations, with some details worth noting:

- The array shows up as **one big anonymous mapping of exactly N MB + 4 KB** (102,400 KB + 4 KB for 100 MB). The extra page holds `malloc`'s bookkeeping header. It is **not** in the small heap region right after the program: glibc's `malloc` serves large requests (over 128 KB by default) with a separate `mmap`, and gives small ones from the heap.
- Its **RSS equals its size**: because the program touches every page, every page is in physical memory. The rest of the address space is tiny and stays the same: a 4 KB code page, the C library, the dynamic loader and a 132 KB stack. Only the array mapping grows with N.
- The addresses show the classic layout: the program's code at the bottom (`0x0000c768…`), libraries and big `mmap` regions much higher up (`0x0000f42b…`), and the stack at the very top (`0x0000ffff…`), growing down toward them.